

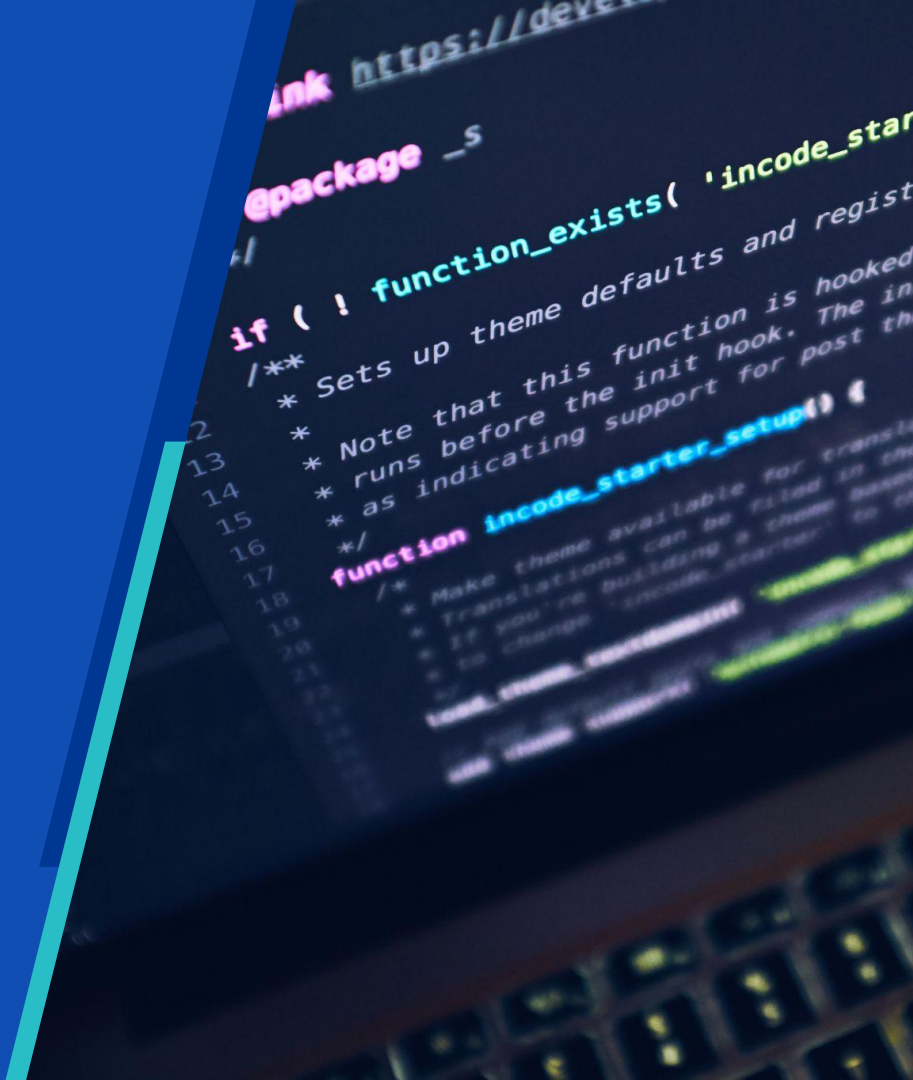
ARKANA



Odoo V18 Technical Training

Prasetiya Mulya University

arkana.co.id



ARKANA

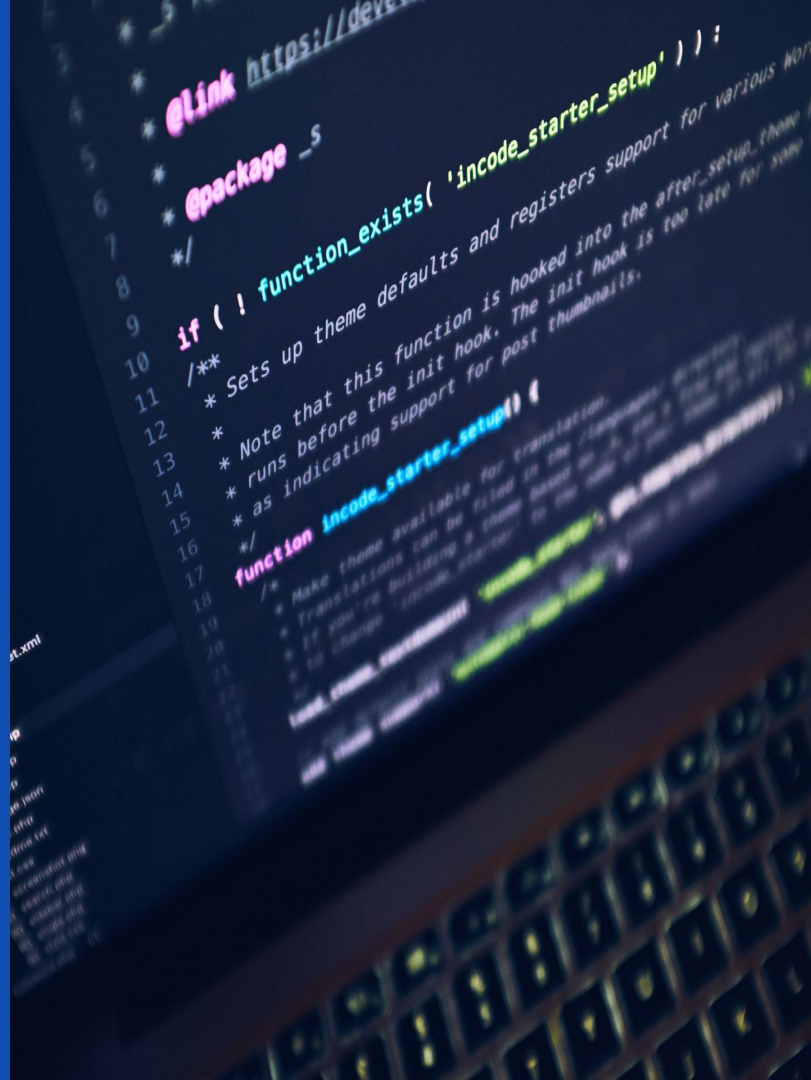


ODOO 18 · TECHNICAL FUNDAMENTAL

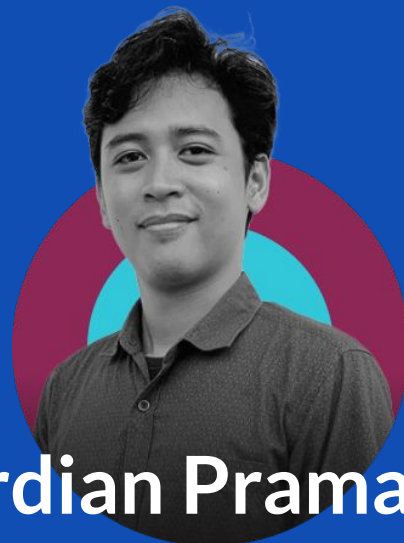
Day 5 — Odoo as REST API Provider

HTTP Controller · Serializer · Response Contract · API Key

Academy Management · studi kasus berkelanjutan dari Day 4 final



Odoo Trainer Technical



Ardian Pramana

Lead Developer @Arkana

Rundown **Training** **Arkana**



- 08.30 - 10.00** : Sesi I
- 10.00 - 10.15** : Coffee Break
- 10.15 - 12.00** : Sesi II
- 11.30 - 13.15** : ISHOMA
- 13.15 - 15.20** : Sesi III
- 15.20 - 15.45** : Break Ashr
- 15.45 - 16.30** : Sesi IV

ARKANA

Follow our **Social Media**



@arkana_academy



@arkana_academy



Arkana Solusi Digital



@arkanasolusidigital

www.arkana.co.id

What We Are Building

GET

`/academy/api/v1/ping`

Health check

A

GET

`/academy/api/v1/courses`

Daftar course (serialized)

B

GET

`/academy/api/v1/courses/<code>`

Course detail + 404

C

POST

`/academy/api/v1/enrollment-requests`

Enrollment draft + API Key

D+E

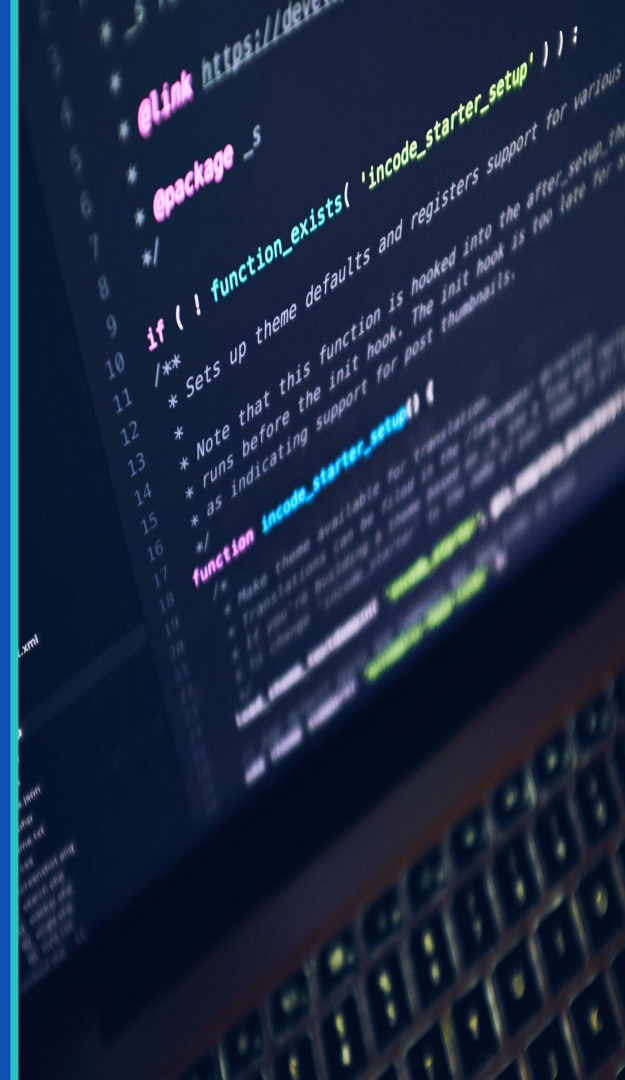


Enrollment selalu DRAFT — approval workflow tetap berjalan. Tidak ada auto-create batch. API tidak bypass ORM.

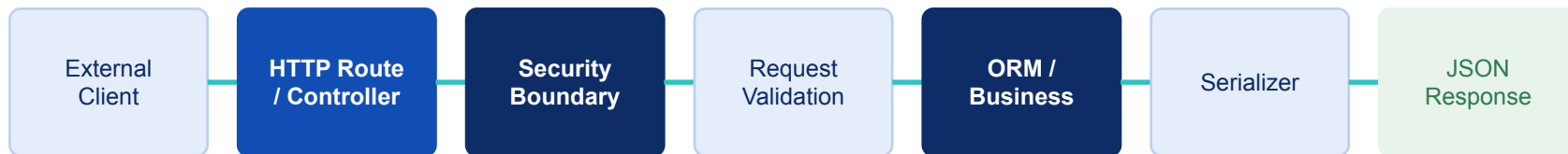
PART 1

How REST Provider Works in Odoo

Odoo model tidak otomatis jadi REST resource



How REST Provider Works in Odoo



- 💡 Controller = API boundary, bukan tempat business logic.
- 💡 Business rules tetap di Model / Workflow. Serializer mengontrol field yang aman.
- 💡 API tidak pernah bypass approval workflow — enrollment selalu dibuat draft.

Controller sebagai API Boundary

✓ Controller BOLEH

- Parse request payload / path params
- Validasi field wajib ada atau tidak
- Lookup record via ORM (env[...])
- Panggil business method di Model
- Serialize response ke JSON dictionary
- Return HTTP status code yang tepat

✗ Controller JANGAN

- Menulis business rule langsung di controller
- Bypass approval workflow
- Auto-confirm enrollment tanpa review
- Return seluruh recordset fields (tanpa filter)
- Hardcode credentials di kode
- Return stack trace ke client

Anatomy of Odoo HTTP Controller

Extend http.Controller

Bukan class biasa

@http.route()

Mendaftarkan URL ke Odoo router

type="http"

Return Response manual, bukan JSON-RPC

auth="public"

Tidak butuh Odoo session (+ sudo di dalam)

request.make_response()

Kontrol penuh atas status & body

```
from odoo import http
from odoo.http import request
import json

class AcademyApiController(http.Controller):

    @http.route(
        "/academy/api/v1/ping",
        type="http",          # ← bukan "json"
        auth="public",
        methods=["GET"],
    )
    def ping(self, **kw):
        payload = {
            "success": True,
            "data": {"status": "ok", "version": "1.0"},
            "error": None,
        }
        return request.make_response(
            json.dumps(payload),

headers=[("Content-Type", "application/json)],
            status=200,
        )
```

Mengapa `type="http"` untuk Lab Ini?

`type="json"` (JSON-RPC)

- Request harus format `{jsonrpc, method, params}`
- Response dibungkus `{result: ..., id: ...}`
- HTTP status selalu 200
- Tidak cocok untuk REST-style API

`type="http"` ← kita pakai ini

- Request: plain HTTP (GET, POST, dll.)
- Response: bebas, kita kontrol penuh
- HTTP status kita tentukan (200, 201, 400, 404...)
- Cocok untuk REST API provider



`type="http" + make_response(json.dumps(...), status=...) = REST API pattern kita hari ini.`

Response Contract

Semua endpoint menggunakan shape yang sama.

HTTP 200 / 201

```
// Success
{
  "success": true,
  "data": { ... },
  "error": null
}
```

HTTP 4xx

```
// Error
{
  "success": false,
  "data": null,
  "error": {
    "code": "ERROR_CODE",
    "message": "Readable message."
  }
}
```

HTTP Status	Kapan dipakai
200 OK	GET berhasil; POST → enrollment sudah ada
201 Created	POST → enrollment baru berhasil dibuat
400 Bad Request	Field wajib tidak ada / JSON tidak valid
401 Unauthorized	API Key salah atau tidak ada

Serializer & Safe Field Exposure

```
def _course_to_dict(self, course):  
    # Only expose safe primitive fields.  
    # description = fields.Html() — excluded.  
    return {  
        "code":          course.code or "",  
        "name":          course.name,  
        "level":         course.level or "",  
        "duration_hours": course.duration_hours,  
    }
```

✓ Whitelist fields secara eksplisit

✓ Method terpisah, bukan inline dict

✓ Return primitive (str, int, bool, float)

✗ Jangan return recordset langsung

✗ Jangan expose Html fields ke JSON

✗ Jangan invent fields yang tidak ada



Serializer adalah "export schema". Definiskan eksplisit — jangan andalkan `read([])` tanpa filter.

Checkpoint A — Ping Endpoint

GET /academy/api/v1/ping

- 1 Buat folder `controllers/` di dalam module `academy_management`
- 2 Buat `controllers/__init__.py`: `from . import api_controller`
- 3 Edit root `__init__.py`: tambah `from . import controllers`
- 4 Buat class `AcademyApiController(http.Controller)`
- 5 `@http.route('/academy/api/v1/ping', type='http', auth='public', methods=['GET'])`
- 6 Return `make_response(json.dumps({"success":True,"data":{...},"error":None}), status=200)`

```
$ curl -s http://localhost:8069/academy/api/v1/ping  
→ {"success": true, "data": {"status": "ok", "version": "1.0"}, "error": null}
```

Checkpoint B — GET Courses List

```
@http.route("/academy/api/v1/courses",
            type="http", auth="public",
            methods=["GET"])
def get_courses(self, **kw):
    courses = request.env["academy.course"] \
        .sudo().search([])
    return _ok([self._course_to_dict(c)
                for c in courses])

def _course_to_dict(self, course):
    return {
        "code":          course.code or "",
        "name":          course.name,
        "level":         course.level or "",
        "duration_hours": course.duration_hours,
    }
```

sudo()

auth='public' → pakai sudo untuk ORM access

_course_to_dict()

Serializer — tidak return recordset langsung

list comprehension

Iterasi semua course, serialize satu per satu

_ok() helper

Return {"success":true,"data":[...],"error":null}

description = Html

Tidak dimasukkan serializer — tidak JSON-safe

Checkpoint C — GET Course Detail + 404

```
@http.route("/academy/api/v1/courses/<string:code>",
            type="http", auth="public",
            methods=["GET"])
def get_course(self, code, **kw):
    course = request.env["academy.course"] \
        .sudo().search(
            [("code", "=", code)], limit=1)

    if not course:
        return _err(
            "COURSE_NOT_FOUND",
            "Course with code '%s' not found." % code,
            status=404,
        )

    return _ok(self._course_to_dict(course))
```

<string:code>

Dynamic route param — Odoo router extract otomatis

search([...], limit=1)

Cari berdasarkan code, limit 1

if not course:

Recordset kosong = falsy — cek ini

_err() helper

Return {"success":false,"data":null,"error":{...}}

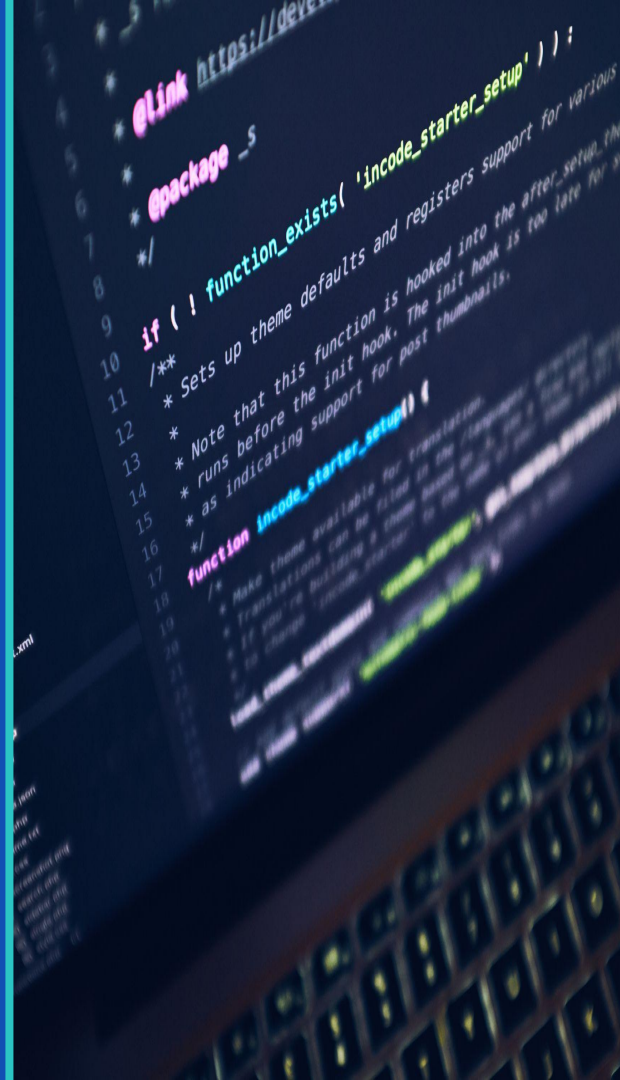
status=404

HTTP 404 = transport; COURSE_NOT_FOUND = aplikasi

PART 2

Business-safe POST Endpoint

Enrollment selalu draft — approval workflow tidak disentuh



Business-safe POST Endpoint

```
POST
/academy/api/v1/enrollment-requests
{
  "student_name": "Budi Santoso",
  "student_email": "budi@example.com",
  "batch_code": "PY-101-JAN",
  "notes": "Weekend batch"
}
```

1

Parse & validate payload

2

Lookup batch by batch_code

3

Find/create student by email

4

Create enrollment → state='draft'

5

Existing → 200 OK | New → 201 Created



Tidak ada auto-create batch. Tidak ada state=confirmed. Tidak ada set approval metadata.

Mengapa batch_code? Tidak Auto-create Batch?

batch_code sebagai External Key

- External system tidak tahu internal Odoo ID
- code adalah identifier stabil yang bisa dishare
- URL-friendly dan human-readable (PY-101-JAN)
- Unique constraint wajib ada di academy.batch.code

```
_sql_constraints = [  
    ("code_unique", "unique(code)",  
     "Batch code must be unique."),  
]
```

Tidak Auto-create Batch

- Batch dibuat melalui proses bisnis (approval, kapasitas)
- API hanya boleh reference batch yang sudah ada
- Jika tidak ditemukan → 404 BATCH_NOT_FOUND
- API tidak punya konteks untuk create batch baru



API yang baik: lookup yang sudah ada, jangan ciptakan data tanpa konteks bisnis. Resource tidak ditemukan → 404.

Checkpoint D — POST Enrollment Request

```
@http.route("/academy/api/v1/enrollment-requests",
    type="http", auth="public",
    methods=["POST"], csrf=False)
def create_enrollment_request(self, **kw):
    try:
        payload = json.loads(
            request.httprequest.data or b"{}")
    except (json.JSONDecodeError, ValueError):
        return _err("INVALID_JSON", "...", status=400)
    required = ["student_name", "student_email", "batch_code"]
    missing = [f for f in required
                if not (payload.get(f) or "").strip()]
    if missing:
        return _err("MISSING_FIELDS", "...", status=400)
    batch = env["academy.batch"].sudo().search(
        [("code", "=", payload["batch_code"])], limit=1)
    if not batch:
        return _err("BATCH_NOT_FOUND", "...", status=404)
    ...create student if needed...
    if existing: return _ok(..., status=200)
    enrollment = env["academy.enrollment"].sudo().create({
        "batch_id": batch.id, "student_id": student.id,
        "notes": notes, # bukan "note"
```

csrf=False

Wajib untuk API — tidak ada browser form

json.loads(...data)

Read raw body, bukan kw params

MISSING_FIELDS

Validate sebelum ORM access

BATCH_NOT_FOUND

Lookup dulu — jangan auto-create

"notes" bukan "note"

academy.enrollment.notes = fields.Text()

201 vs 200

Baru=201 Created; Existing=200 OK

Checkpoint E — API Key Boundary

POST request → Header X-API-Key → Validated → Proceed (or 401)

```
def _check_api_key(self):
    # Key name : academy_management.api_key
    # Demo value: academy-demo-key
    #           (seeded in data/academy_api_data.xml)
    incoming = request.httprequest.headers.get(
        "X-API-Key", "")
    stored = (
        request.env["ir.config_parameter"]
        .sudo()
        .get_param("academy_management.api_key", "")
    )
    return bool(incoming and incoming == stored)

# In create_enrollment_request() - first line:
if not self._check_api_key():
    return _err("UNAUTHORIZED",
```

⚠ Shared Key
Tidak terikat user Odoo spesifik

⚠ No Audit Log
Tidak ada trail siapa yang memanggil

⚠ Plain Comparison
Tidak ada hashing / timing-safe

✓ Cukup untuk Lab
Pola boundary — bukan prod auth



auth='public' + sudo() — Pahami Risikonya

Aspek	Artinya	Risiko
auth='public'	Request tidak butuh session Odoo	Siapapun bisa akses URL — belum ada gate
sudo()	ORM berjalan dengan hak superuser	Bypass Odoo access rights — berbahaya tanpa validasi
Kombinasi keduanya	Tidak perlu login, akses semua data	Production: WAJIB punya gate (API Key, OAuth)
Lab context	Training simplification	Di produksi: gunakan user-scoped API key / OAuth2

Production Direction:

- Gunakan res.users.apikeys (Odoo native) atau OAuth2
- Scope akses per user / partner — bukan shared superuser

- Log semua request (siapa, kapan, apa yang diakses)
- Rate limit di layer Nginx/reverse proxy

Postman Test Plan

1	GET	GET Ping	200 success:true, data.status:ok
2	GET	GET Courses (all)	200 data:[...]
3	GET	GET Course (valid code)	200 course object
4	GET	GET Course (404)	404 COURSE_NOT_FOUND
5	POST	POST Enrollment (valid)	201 state:draft
6	POST	POST same payload again	200 'Enrollment already exists.'
7	POST	POST Missing field	400 MISSING_FIELDS
8	POST	POST Wrong API Key	401 UNAUTHORIZED
9	POST	POST No API Key header	401 UNAUTHORIZED
10	POST	POST Batch not found	404 BATCH_NOT_FOUND
11	POST	POST Invalid JSON	400 INVALID_JSON

Kesalahan yang Sering Terjadi

✗ Lupa `"from . import controllers"` di `__init__.py`

✓ Tambahkan baris tersebut — route tidak terdaftar tanpa ini

✗ `type="json"` di route

✓ Pakai `type="http"` agar bisa kontrol HTTP status code

✗ `json.dumps(courses)` langsung

✓ Recordset tidak serializable — selalu lewat `_course_to_dict()`

✗ Lupa `csrf=False` di POST route

✓ Wajib untuk API — tanpa ini POST selalu 403

✗ Pakai `"note"` di enrollment create

✓ `academy.enrollment` punya `"notes"` (`fields.Text()`) — bukan `"note"`

✗ Auto-create batch jika tidak ditemukan

✓ Return `BATCH_NOT_FOUND 404` — jangan create batch

✗ Set `state="confirmed"` di POST

✓ Hanya default `"draft"` — approval workflow di Odoo UI

✗ Hardcode API key di source code

✓ Baca dari `ir.config_parameter` via `get_param()`

Production Awareness — Dari Lab ke Prod

Kode lab ini mendemo pola. Sebelum ke produksi:

Area	Lab (Sekarang)	Production Direction
Authentication	X-API-Key via <code>ir.config_parameter</code>	<code>res.users.apikeys</code> atau OAuth2
Access Control	<code>auth='public' + sudo()</code> global	Scope per user, <code>with_user(user)</code>
Error Handling	Basic try/except	Structured logging, no stack trace
Input Validation	Manual field check	Schema validator (Cerberus, dll.)
Rate Limiting	Tidak ada	Nginx <code>limit_req</code>
Audit Trail	Tidak ada	Log model / chatter / SIEM

Academy dengan Feature REST API

A Controller Boundary

`http.Controller` memisahkan HTTP dari ORM. Bukan tempat bisnis logic.

B Validasi Sebelum ORM

Cek field, cek resource — sebelum menyentuh database.

C Serializer Eksplisit

`_course_to_dict()` — hanya field aman yang keluar. Tidak return recordset.

D Response Contract

`{success, data, error}` — semua endpoint shape yang sama.

E Security Boundary (Demo)

X-API-Key via `academy_management.api_key`.
Production butuh lebih.

F Approval Workflow Aman

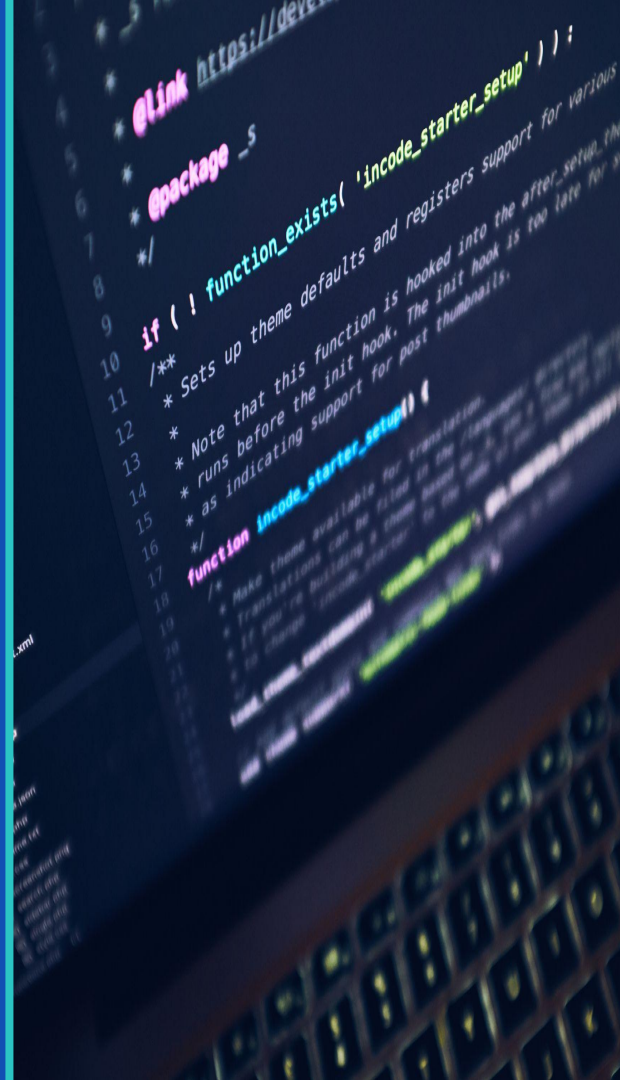
Enrollment dibuat `state='draft'`. API tidak bypass approval Odoo.

Semua endpoint berjalan di atas approval workflow Day 3 final — API tidak pernah bypass. 🎉

PART 3

Deployment

Panduan deployment untuk environment Prasmul



Satu fitur melewati tiga lingkungan



Pilih entitas sebelum menentukan branch dan database.



Feedback selalu kembali ke feature branch yang sama.



Tidak ada direct push ke sandbox atau production.

Satu repository, dua jalur deployment

Tentukan entitas sebelum coding—jangan mencampur branch dan database.

Area	Universitas	ELI
Branch awal	production_univ	production
Local DB	Salinan DB Universitas	Salinan DB ELI
Sandbox	sandbox-univ	Belum dikonfirmasi
Production	production_univ	production
Server owner	Tim Prasmul	Tim Prasmul
Status	Jalur siap digunakan	Konfirmasi sandbox

 **Branch 18.0*-staging-support adalah target internal Arkana, bukan target PR tim Prasmul.**

Contoh: fitur baru untuk Universitas

```
git switch production_univ  
git pull  
git switch -c feature-xyz  
  
Local DB: v18_prasmul_univ_dev  
Sandbox: sandbox-univ  
Production: production_univ
```

1

Buat feature-xyz dari production_univ

2

Coding dan tes awal pada DB local Univ

3

PR feature-xyz → sandbox-univ

4

Perbaiki branch sampai testing disetujui

5

PR feature-xyz → production_univ



Reviewer Arkana yang melakukan merge—peserta tidak direct push.

Triage: kontribusi tanpa akses merge

✓ PESERTA BOLEH

- Membuat fork dan feature branch
- Mengembangkan dan menguji di local
- Membuka PR ke sandbox
- Menindaklanjuti feedback di branch
- Melampirkan hasil testing
- Membuka PR production setelah disetujui

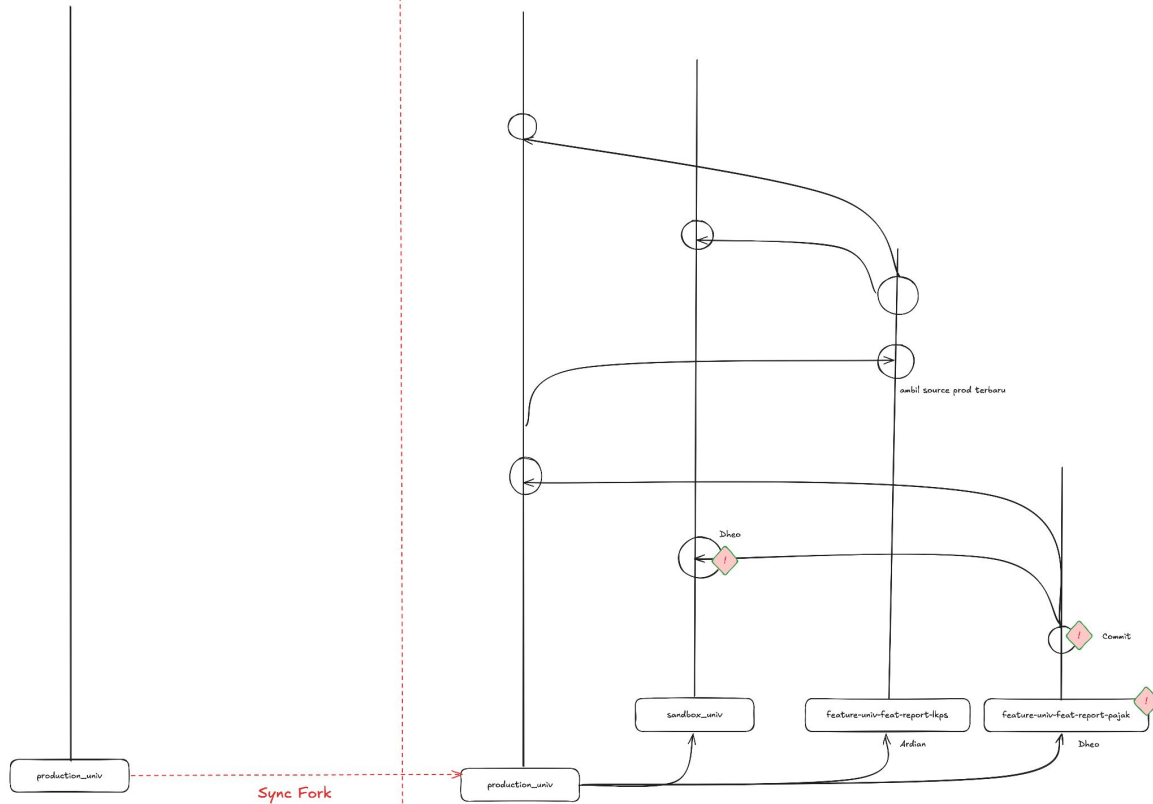
✗ PESERTA JANGAN

- Direct push ke branch utama
- Merge pull request sendiri
- Mencampur perubahan Univ dan ELI
- Mengganti target branch tanpa review
- Menguji dengan database entitas yang salah
- Deploy di luar proses yang disepakati

Arkana repo

UNIV Repo

Corat-coret alur kerja git branching



Catatan:
1. Database sandbox sebaiknya sering di refresh dari prod
2. Beberapa modul yang terlanjur masuk sandbox tapi di di refresh itu perlu upgrade
3.

Q & A

Terima kasih telah menyelesaikan sesi development Training.

Terima Kasih

Prasetiya Mulya University · Odoo V18 Technical Training

ARKANA

www.arkana.co.id